

Texto gerado pelo GEMINI

O objetivo é demonstrar o ciclo de vida da programação assistida por IA:

Construção → Refatoração → Validação.

1. Construção da Base (O papel do "Copiloto")

O primeiro passo, conforme o guia de Engenharia de Prompt, é não ser genérico.

Em vez de pedir "uma calculadora", estruturamos o pedido para que a IA atue como um desenvolvedor sênior.

Prompt utilizado:

"Atue como um desenvolvedor Python Sênior. Crie a lógica central de uma calculadora que realize as quatro operações básicas. O código deve ser modular e incluir o tratamento de erro para divisão por zero."

Código Gerado:

Python

```
def calculadora(a, b, operacao):  
    if operacao == 'soma':  
        return a + b  
    elif operacao == 'subtracao':  
        return a - b  
    elif operacao == 'multiplicacao':  
        return a * b  
    elif operacao == 'divisao':  
        if b == 0:  
            return "Erro: Divisão por zero"  
        return a / b
```

2. Refatoração Inteligente (O "Padrão Pythônico")

Seguindo o material, o papel do humano é elevar o código. Pedimos à IA para refatorar o código acima para torná-lo mais performático, usando dicionários (mapeamento) em vez de múltiplos if/else, e adicionando segurança nas entradas.

Código Refatorado:

Python

```
def executar_calculadora():

    # Dicionário de operações para evitar múltiplos ifs (mais limpo e rápido)

    operacoes = {

        '+': lambda x, y: x + y,

        '-': lambda x, y: x - y,

        '*': lambda x, y: x * y,

        '/': lambda x, y: x / y if y != 0 else "Erro: Divisão por zero"

    }

    try:

        n1 = float(input("Primeiro número: "))

        n2 = float(input("Segundo número: "))

        op = input("Operação (+, -, *, /): ")

        # Busca a operação no dicionário ou retorna erro de operação inválida

        resultado = operacoes.get(op, lambda x, y: "Operação inválida")(n1, n2)

        print(f"Resultado: {resultado}")

    except ValueError:

        print("Erro: Entrada inválida. Digite apenas números.")

executar_calculadora()
```

3. Análise Crítica (A sua responsabilidade como Tech Lead)

Para concluir o que a atividade pede, você deve validar o resultado final com base nos critérios de sucesso das aulas:

1. **Redução da Carga Cognitiva:** A IA cuidou da sintaxe do dicionário e da lógica da lambda, permitindo que você focasse na estrutura do programa.
2. **Segurança e Tratamento de Erros:** O código agora previne que o programa "quebre" se o usuário digitar uma letra (ValueError) ou se tentar dividir por zero.
3. **Verificação de "Alucinações":** Como revisor, você garante que a lógica da divisão por zero foi implementada corretamente antes de integrar o código ao projeto.

4. Resumo das Ferramentas Recomendadas

Para testar este código agora mesmo, os materiais sugerem:

- **Replit:** Para ver o erro e a correção em tempo real.
- **VS Code com GitHub Copilot:** Para sentir a aceleração de 55% no desenvolvimento através do autocompletar.
- **Google Colab:** Para documentar o passo a passo da lógica.

Conclusão: Você utilizou a IA como um **amplificador cognitivo**. A máquina entregou a velocidade e a sugestão de padrões, mas você, como programador, deu a direção e validou a segurança.